

從原型設計到正式環境：在 Vertex AI 執行分散式訓練

來源：<https://codelabs.developers.google.com/vertex-p2p-distributed?hl=zh-tw>

步驟數：7

目錄

1. [1. 總覽](#)
2. [2. Vertex AI 簡介](#)
3. [3. 分散式訓練總覽](#)
4. [4. 設定環境](#)
5. [5. 單一機器，多 GPU 訓練](#)
6. [6. \[選用\] 多工作站訓練](#)
7. [7. 清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將使用 [Vertex AI](#)，透過 TensorFlow 在 Vertex AI 訓練 上執行分散式訓練工作。

本實驗室是「從原型設計到投入正式環境」系列影片的一部分。請務必先完成[先前的實驗室](#)，再進行這個實驗室。如要瞭解詳情，請觀看隨附的影片系列：



。

課程內容

內容如下：

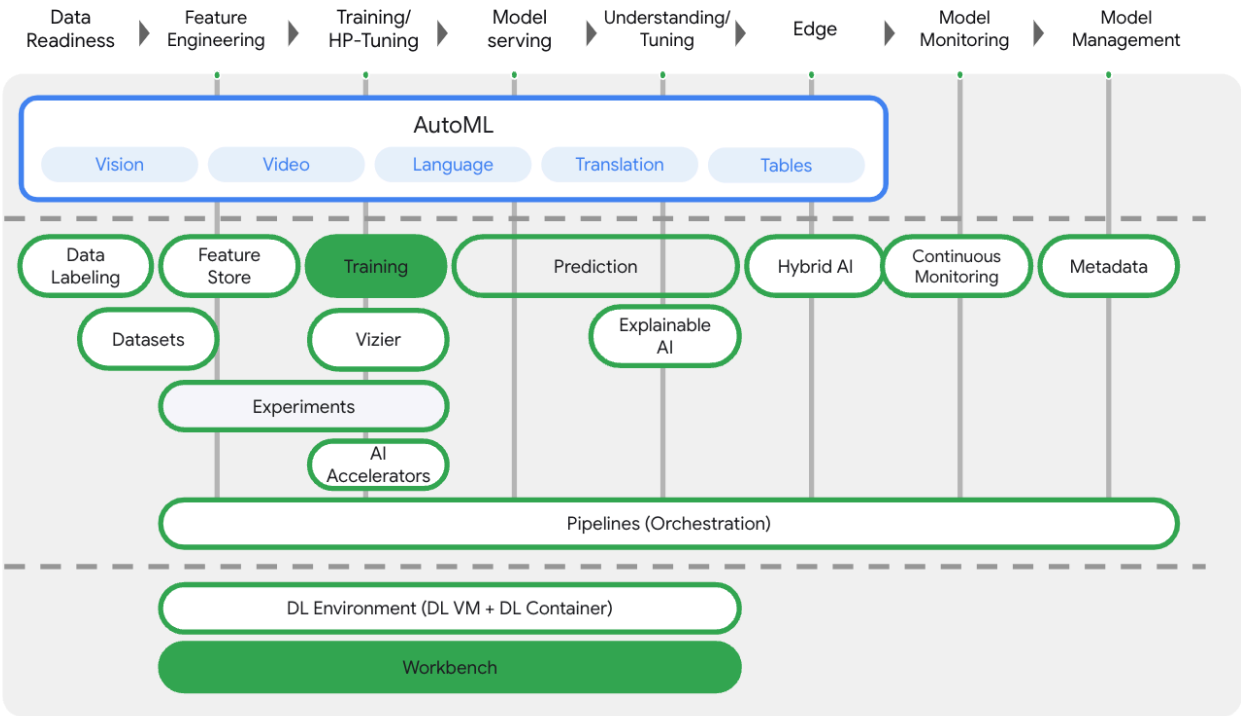
- 在單一機器上使用多個 GPU 執行分散式訓練
- 在多部機器上執行分散式訓練

在 Google Cloud 上執行這個實驗室的總費用約為 **\$2 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。Vertex AI 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於下列產品：訓練和 Workbench



步驟 3 · 約 0 分鐘

3. 分散式訓練總覽

如果您只有一個 GPU，TensorFlow 會使用這個加速器加快模型訓練速度，您不需要額外進行任何作業。不過，如果想透過使用多個 GPU 獲得額外效能提升，則需要使用 `tf.distribute`，這是 TensorFlow 的模組，可跨多個裝置執行運算。

本實驗室的第一節會使用 `tf.distribute.MirroredStrategy`，您只要變更幾行程式碼，就能將其新增至訓練應用程式。這項策略會在機器上的每個 GPU 建立模型副本。後續的梯度更新會以同步方式進行。也就是說，每個 GPU 會針對不同的輸入資料片段，透過模型計算正向和反向傳遞。然後，系統會透過稱為「全縮減」的程序，彙整所有 GPU 的計算梯度並取平均值。系統會使用這些平均梯度更新模型參數。

實驗室結尾的選用章節會使用 `tf.distribute.MultiWorkerMirroredStrategy`，這與 `MirroredStrategy` 類似，但適用於多部機器。每部機器也可能有多個 GPU。例如，`MirroredStrategy`、`MultiWorkerMirroredStrategy` 是同步資料平行化策略，您只需稍微修改程式碼即可使用。從單一機器的同步資料平行處理遷移至多部機器時，主要差異在於每個步驟結束時的梯度，現在需要跨機器中的所有 GPU 和叢集中的所有機器同步處理。

您不需要瞭解詳細資料，也能完成本實驗室，但如果想進一步瞭解 TensorFlow 中的分散式訓練運作方式，請觀看下方影片：



步驟 4 · 約 10 分鐘

4. 設定環境

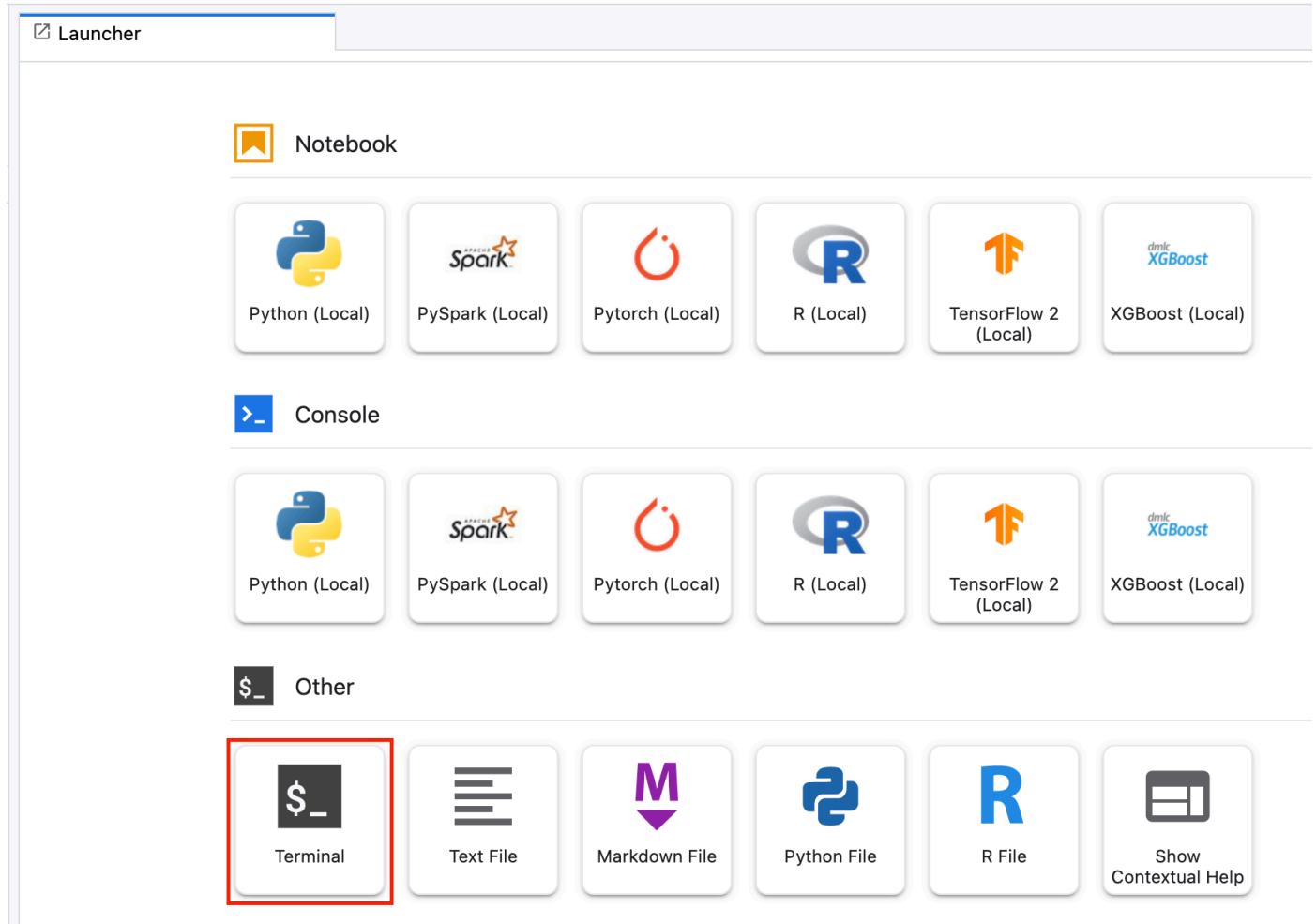
完成「[使用 Vertex AI 訓練自訂模型](#)」實驗室中的步驟，設定環境。

步驟 5 · 約 25 分鐘

5. 單一機器，多 GPU 訓練

您需要將訓練應用程式程式碼放入 [Docker 容器](#)，並將這個容器推送至 [Google Artifact Registry](#)，才能將分散式訓練工作提交至 Vertex AI。使用這種方法，您可以訓練以任何架構建構的模型。

首先，請從先前實驗室建立的 Workbench 筆記本的啟動器選單中，開啟終端機視窗。



步驟 1：編寫訓練程式碼

建立名為 `flowers-multi-gpu` 的新目錄，然後切換至該目錄：

```
mkdir flowers-multi-gpu
cd flowers-multi-gpu
```

執行下列指令，為訓練程式碼建立目錄，以及您將在其中新增下列程式碼的 Python 檔案。

```
mkdir trainer
touch trainer/task.py
```

`flowers-multi-gpu/` 目錄現在應該包含下列項目：

```
+ trainer/  
  + task.py
```

接著，開啟剛才建立的 `task.py` 檔案，然後複製下列程式碼。

請將 `BUCKET_ROOT` 中的 `{your-gcs-bucket}` 改成您在[實驗室 1](#) 中儲存花卉資料集的 Cloud Storage bucket。


```

import tensorflow as tf
import numpy as np
import os

## Replace {your-gcs-bucket} !!
BUCKET_ROOT='/gcs/{your-gcs-bucket}'

# Define variables
NUM_CLASSES = 5
EPOCHS=10
BATCH_SIZE = 32

IMG_HEIGHT = 180
IMG_WIDTH = 180

DATA_DIR = f'{BUCKET_ROOT}/flower_photos'

def create_datasets(data_dir, batch_size):
    '''Creates train and validation datasets.'''

    train_dataset = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset="training",
        seed=123,
        image_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=batch_size)

    validation_dataset = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset="validation",
        seed=123,
        image_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=batch_size)

    train_dataset = train_dataset.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
    validation_dataset = validation_dataset.cache().prefetch(buffer_size=tf.data.AUTOTUNE)

    return train_dataset, validation_dataset

def create_model():
    '''Creates model.'''

    model = tf.keras.Sequential([
        tf.keras.layers.Resizing(IMG_HEIGHT, IMG_WIDTH),
        tf.keras.layers.Rescaling(1./255, input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
        tf.keras.layers.Conv2D(16, 3, padding='same', activation='relu'),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, padding='same', activation='relu'),
        tf.keras.layers.MaxPooling2D(),

```

```

    tf.keras.layers.Conv2D(64, 3, padding='same', activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(NUM_CLASSES, activation='softmax')
])
return model

def main():

    # Create distribution strategy
    strategy = tf.distribute.MirroredStrategy()

    # Get data
    GLOBAL_BATCH_SIZE = BATCH_SIZE * strategy.num_replicas_in_sync
    train_dataset, validation_dataset = create_datasets(DATA_DIR, BATCH_SIZE)

    # Wrap model creation and compilation within scope of strategy
    with strategy.scope():
        model = create_model()
        model.compile(optimizer=tf.keras.optimizers.Adam(),
                      loss=tf.keras.losses.SparseCategoricalCrossentropy(),
                      metrics=['accuracy'])

    history = model.fit(
        train_dataset,
        validation_data=validation_dataset,
        epochs=EPOCHS
    )

    model.save(f'{BUCKET_ROOT}/model_output')

if __name__ == "__main__":
    main()

```

建構容器前，請先深入瞭解程式碼。使用分散式訓練時，有幾個專屬元件。

- 在 `main()` 函式中，系統會建立 `MirroredStrategy` 物件。接著，您要在策略範圍內包裝模型變數的建立作業。這個步驟會告知 TensorFlow 哪些變數應在 GPU 間鏡像處理。
- 批次大小會依 `num_replicas_in_sync` 放大。在 TensorFlow 中使用同步資料平行處理策略時，建議您調整批量大小。如要瞭解詳情，請參閱[這篇文章](#)。

步驟 2：建立 Dockerfile

如要將程式碼容器化，您需要建立 Dockerfile。Dockerfile 包含執行映像檔所需的所有指令。這會安裝所有必要的程式庫，並設定訓練程式碼的進入點。

在終端機中，於 `flowers` 目錄的根目錄建立空白的 Dockerfile：

```
touch Dockerfile
```

`flowers-multi-gpu/` 目錄現在應該包含下列項目：

```
+ Dockerfile
+ trainer/
  + task.py
```

開啟 `Dockerfile`，然後將下列內容複製到其中：

```
FROM gcr.io/deeplearning-platform-release/tf2-gpu.2-8

WORKDIR /

# Copies the trainer code to the docker image.
COPY trainer /trainer

# Sets up the entry point to invoke the trainer.
ENTRYPOINT ["python", "-m", "trainer.task"]
```

步驟 3：建構容器

在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

在 Artifact Registry 中建立存放區。我們會使用在第一個實驗室中建立的存放區。

```
REPO_NAME='flower-app'
```

使用 Artifact Registry 中容器映像檔的 URI 定義變數：

```
IMAGE_URI=us-central1-
docker.pkg.dev/$PROJECT_ID/$REPO_NAME/flower_image_distributed:single_machine
```

設定 Docker

```
gcloud auth configure-docker \
  us-central1-docker.pkg.dev
```

接著，從 `flowers-multi-gpu` 目錄的根層級執行下列指令，建構容器：

```
docker build ./ -t $IMAGE_URI
```

最後，將其推送至 Artifact Registry：

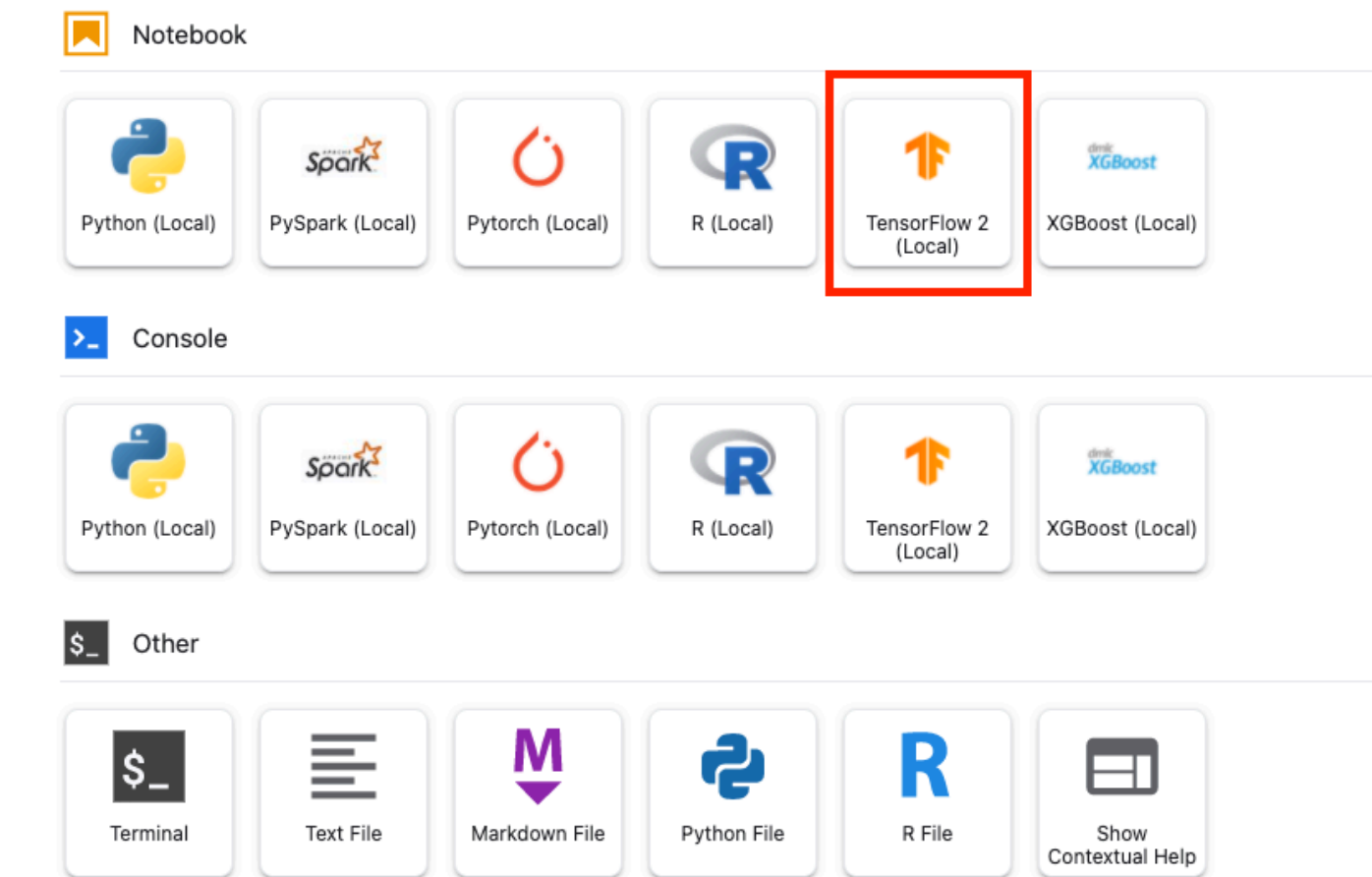
```
docker push $IMAGE_URI
```

容器已推送至 Artifact Registry，現在可以啟動訓練工作。

步驟 4：使用 SDK 執行作業

在本節中，您將瞭解如何使用 Vertex AI Python SDK 設定及啟動分散式訓練工作。

從 Launcher 建立 TensorFlow 2 筆記本。



匯入 Vertex AI SDK。

```
from google.cloud import aiplatform
```

然後定義 CustomContainerTrainingJob 。

您需要將 `container_uri` 中的 `{PROJECT_ID}`，以及 `staging_bucket` 中的 `{YOUR_BUCKET}` 替換為實際值。

[illegible]

定義工作後，即可執行工作。您將加速器數量設為 2。如果只使用 1 個 GPU，這不會視為分散式訓練。在單一機器上進行分散式訓練時，您會使用 2 個以上的加速器。

```
my_custom_job.run(replica_count=1,
                  machine_type='n1-standard-4',
                  accelerator_type='NVIDIA_TESLA_V100',
                  accelerator_count=2)
```

您可以在控制台中查看工作進度。

Vertex AI

Dashboard

Datasets

Features

Labelling tasks

Workbench

Pipelines

Training

Experiments

Models

Training

+ CREATE

TRAINING PIPELINES

CUSTOM JOBS

HYPERPARAMETER TUNING JOBS

Custom jobs specify how Vertex AI runs your custom training code, including worker pools, machine types and settings related to your Python training application and custom container. Custom jobs are only used by custom-trained models and not AutoML models.

[Learn more](#)

Region

us-central1 (Iowa)

?

Filter

Enter a property name

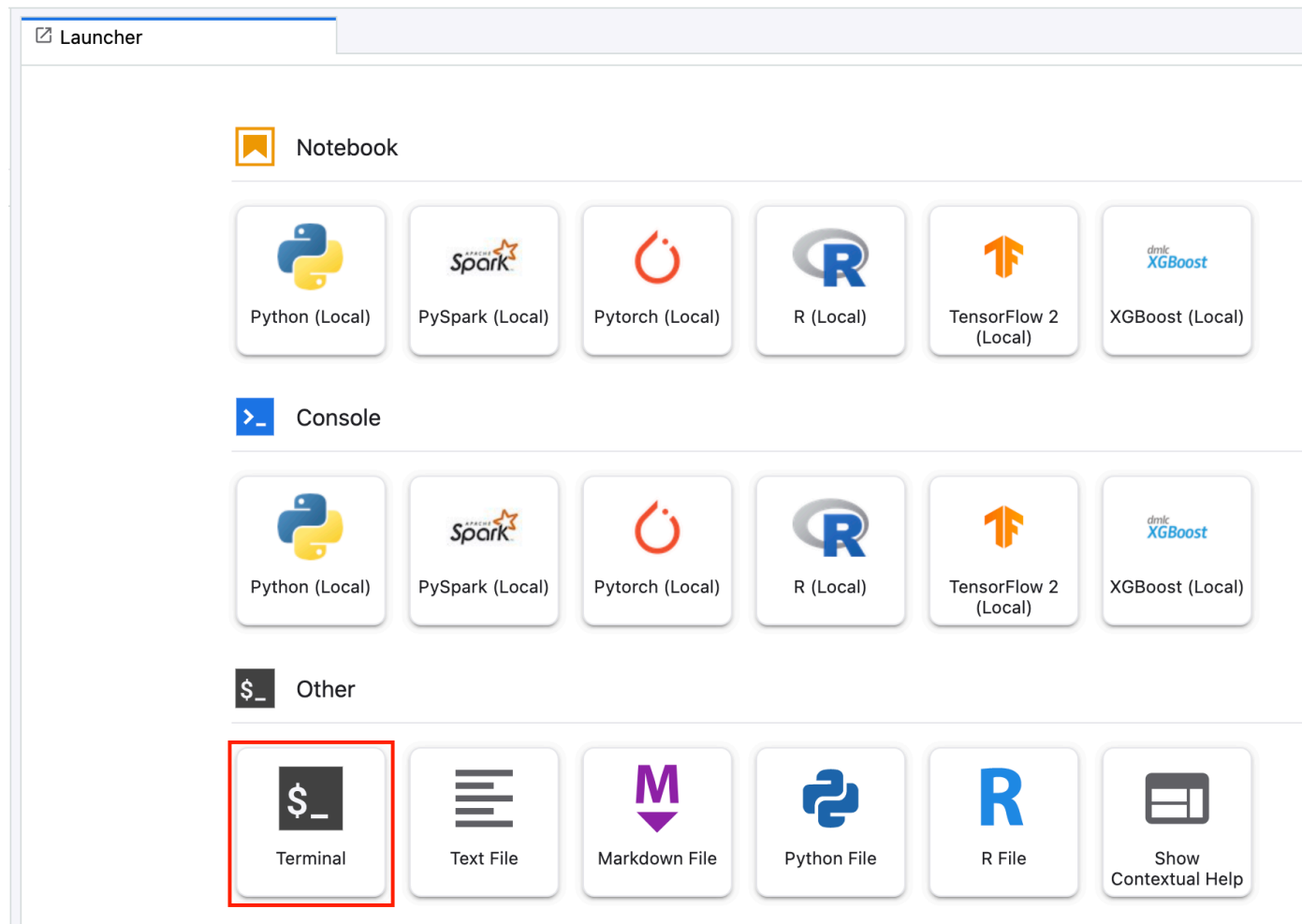
Name	ID	Status	Job type	Model type
flowers-multi-gpu-custom-job	5112238961850318848	Pending	Custom job	—

步驟 6 · 約 25 分鐘

6. [選用] 多工作站訓練

您已在單一機器上使用多個 GPU 試過分散式訓練，現在可以跨多部機器訓練，將分散式訓練技能提升到下一個層次。為降低成本，我們不會在這些機器中新增任何 GPU，但您可以視需要新增 GPU，進行實驗。

在筆記本執行個體中開啟新的終端機視窗：



步驟 1：編寫訓練程式碼

建立名為 `flowers-multi-machine` 的新目錄，然後切換至該目錄：

```
mkdir flowers-multi-machine
cd flowers-multi-machine
```

執行下列指令，為訓練程式碼建立目錄，以及您將在其中新增下列程式碼的 Python 檔案。

```
mkdir trainer
touch trainer/task.py
```

`flowers-multi-machine/` 目錄現在應該包含下列項目：

```
+ trainer/  
  + task.py
```

接著，開啟剛才建立的 `task.py` 檔案，然後複製下列程式碼。

請將 `BUCKET_ROOT` 中的 `{your-gcs-bucket}` 改成您在[實驗室 1](#) 中儲存花卉資料集的 Cloud Storage bucket。

```

import tensorflow as tf
import numpy as np
import os

## Replace {your-gcs-bucket} !!
BUCKET_ROOT='/gcs/{your-gcs-bucket}'

# Define variables
NUM_CLASSES = 5
EPOCHS=10
BATCH_SIZE = 32

IMG_HEIGHT = 180
IMG_WIDTH = 180

DATA_DIR = f'{BUCKET_ROOT}/flower_photos'
SAVE_MODEL_DIR = f'{BUCKET_ROOT}/multi-machine-output'

def create_datasets(data_dir, batch_size):
    '''Creates train and validation datasets.'''

    train_dataset = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset="training",
        seed=123,
        image_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=batch_size)

    validation_dataset = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset="validation",
        seed=123,
        image_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=batch_size)

    train_dataset = train_dataset.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
    validation_dataset = validation_dataset.cache().prefetch(buffer_size=tf.data.AUTOTUNE)

    return train_dataset, validation_dataset

def create_model():
    '''Creates model.'''

    model = tf.keras.Sequential([
        tf.keras.layers.Resizing(IMG_HEIGHT, IMG_WIDTH),
        tf.keras.layers.Rescaling(1./255, input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
        tf.keras.layers.Conv2D(16, 3, padding='same', activation='relu'),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, padding='same', activation='relu'),

```



```

    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64, 3, padding='same', activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(NUM_CLASSES, activation='softmax')
])
return model

def _is_chief(task_type, task_id):
    '''Helper function. Determines if machine is chief.'''

    return task_type == 'chief'

def _get_temp_dir(dirpath, task_id):
    '''Helper function. Gets temporary directory for saving model.'''

    base_dirpath = 'workertemp_' + str(task_id)
    temp_dir = os.path.join(dirpath, base_dirpath)
    tf.io.gfile.makedirs(temp_dir)
    return temp_dir

def write_filepath(filepath, task_type, task_id):
    '''Helper function. Gets filepath to save model.'''

    dirpath = os.path.dirname(filepath)
    base = os.path.basename(filepath)
    if not _is_chief(task_type, task_id):
        dirpath = _get_temp_dir(dirpath, task_id)
    return os.path.join(dirpath, base)

def main():
    # Create distribution strategy
    strategy = tf.distribute.MultiWorkerMirroredStrategy()

    # Get data
    GLOBAL_BATCH_SIZE = BATCH_SIZE * strategy.num_replicas_in_sync
    train_dataset, validation_dataset = create_datasets(DATA_DIR, BATCH_SIZE)

    # Wrap variable creation within strategy scope
    with strategy.scope():
        model = create_model()
        model.compile(optimizer=tf.keras.optimizers.Adam(),
                      loss=tf.keras.losses.SparseCategoricalCrossentropy(),
                      metrics=['accuracy'])

    history = model.fit(
        train_dataset,
        validation_data=validation_dataset,
        epochs=EPOCHS

```

```

)

# Determine type and task of the machine from
# the strategy cluster resolver
task_type, task_id = (strategy.cluster_resolver.task_type,
                      strategy.cluster_resolver.task_id)

# Based on the type and task, write to the desired model path
write_model_path = write_filepath(SAVE_MODEL_DIR, task_type, task_id)
model.save(write_model_path)

if __name__ == "__main__":
    main()

```

建構容器前，請先深入瞭解程式碼。程式碼中有幾個元件是訓練應用程式與 `MultiWorkerMirroredStrategy` 搭配運作時的必要元件。

- 在 `main()` 函式中，系統會建立 `MultiWorkerMirroredStrategy` 物件。接著，您要在策略範圍內包裝模型變數的建立作業。這項重要步驟會告知 TensorFlow 哪些變數應在副本之間鏡像處理。
- 批次大小會依 `num_replicas_in_sync` 放大。在 TensorFlow 中使用同步資料平行處理策略時，調整批量是最佳做法。
- 在多個 worker 的情況下，儲存模型會稍微複雜，因為每個 worker 的目的地都必須不同。主要工作站會儲存到所需模型目錄，其他工作站則會將模型儲存到暫時目錄。為避免多個 worker 寫入相同位置，這些暫時目錄必須是專屬目錄。儲存作業可以包含集體作業，也就是所有 worker 都必須儲存，而不只是主 worker。`_is_chief()`、`_get_temp_dir()`、`write_filepath()` 函式和 `main()` 函式都包含樣板程式碼，有助於儲存模型。

請注意，如果您曾在其他環境中使用 `MultiWorkerMirroredStrategy`，可能已設定 `TF_CONFIG` 環境變數。Vertex AI 會自動為您設定 `TF_CONFIG`，因此您不需要在叢集中的每部機器上定義這個變數。

步驟 2：建立 Dockerfile

如要將程式碼容器化，您需要建立 Dockerfile。Dockerfile 包含執行映像檔所需的所有指令。這會安裝所有必要的程式庫，並設定訓練程式碼的進入點。

在終端機中，於 `flowers` 目錄的根目錄建立空白的 Dockerfile：

```
touch Dockerfile
```

`flowers-multi-machine/` 目錄現在應該包含下列項目：

```

+ Dockerfile
+ trainer/
  + task.py

```

開啟 Dockerfile，然後將下列內容複製到其中：

```
FROM gcr.io/deeplearning-platform-release/tf2-gpu.2-8

WORKDIR /

# Copies the trainer code to the docker image.
COPY trainer /trainer

# Sets up the entry point to invoke the trainer.
ENTRYPOINT ["python", "-m", "trainer.task"]
```

步驟 3：建構容器

在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

在 Artifact Registry 中建立存放區。我們會使用在第一個實驗室中建立的存放區。

```
REPO_NAME='flower-app'
```

在 Google Artifact Registry 中，使用容器映像檔的 URI 定義變數：

```
IMAGE_URI=us-central1-  
docker.pkg.dev/$PROJECT_ID/$REPO_NAME/flower_image_distributed:multi_machine
```

設定 Docker

```
gcloud auth configure-docker \  
    us-central1-docker.pkg.dev
```

接著，從 `flowers-multi-machine` 目錄的根層級執行下列指令，建構容器：

```
docker build ./ -t $IMAGE_URI
```

最後，將其推送至 Artifact Registry：

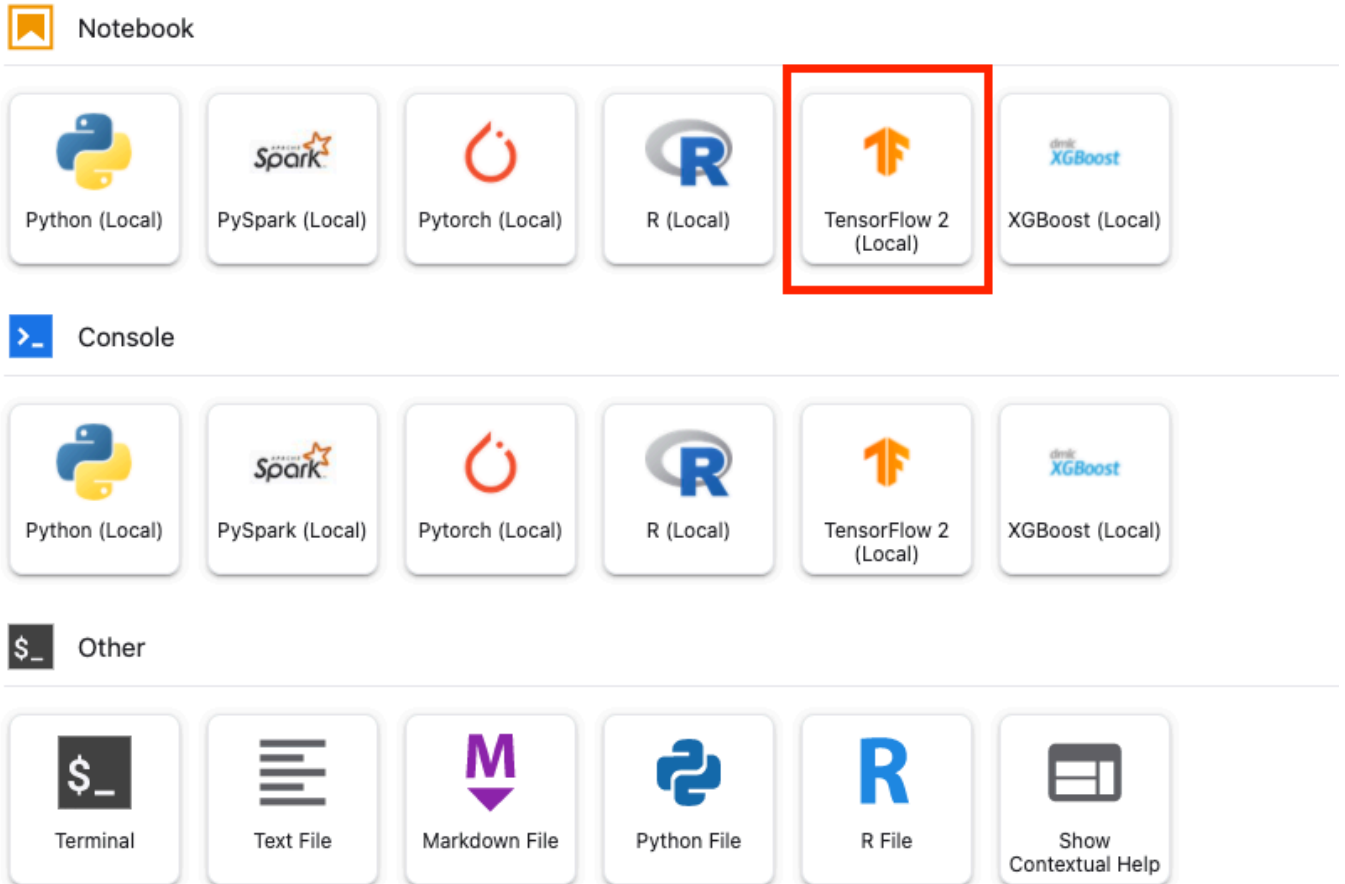
```
docker push $IMAGE_URI
```

容器已推送至 Artifact Registry，現在可以啟動訓練工作。

步驟 4：使用 SDK 執行作業

在本節中，您將瞭解如何使用 Vertex AI Python SDK 設定及啟動分散式訓練工作。

從 Launcher 建立 TensorFlow 2 筆記本。



匯入 Vertex AI SDK。

```
from google.cloud import aiplatform
```

然後定義 `worker_pool_specs`。

Vertex AI 提供 4 個工作人員集區，可涵蓋不同類型的機器工作。

工作站集區 0 會設定主要、主要、排程器或「主要」執行個體。在 `MultiWorkerMirroredStrategy` 中，所有機器都會指定為工作站，也就是執行複製運算的實體機器。除了每部機器都是工作站之外，還需要一個工作站來處理一些額外工作，例如儲存檢查點，以及將摘要檔案寫入 TensorBoard。這部機器稱為「首長」。主要工作站一律只有一個，因此工作站集區 0 的工作站數量一律為 1。

您可以在工作站集區 1 中設定叢集的額外工作站。

`worker_pool_specs` 清單中的第一個字典代表 Worker 集區 0，第二個字典則代表 Worker 集區 1。在本範例中，這兩個設定完全相同。不過，如要在 3 部機器上訓練模型，請將 `replica_count` 設為 2，在工作站集區 1 中新增工作站。如要新增 GPU，您必須在兩個工作站集區的 `machine_spec` 中新增 `accelerator_type` 和 `accelerator_count` 引數。請注意，如要搭配 `MultiWorkerMirroredStrategy` 使用 GPU，叢集中的每部機器必須有相同數量的 GPU。否則工作會失敗。

您必須在 `image_uri` 中取代 `{PROJECT_ID}`。

```
# The spec of the worker pools including machine type and Docker image
# Be sure to replace PROJECT_ID in the "image_uri" with your project.

worker_pool_specs=[
    {
        "replica_count": 1,
        "machine_spec": {
            "machine_type": "n1-standard-4",
        },
        "container_spec": {"image_uri": "us-central1-docker.pkg.dev/{PROJECT_ID}/flower-
app/flower_image_distributed:multi_machine"}
    },
    {
        "replica_count": 1,
        "machine_spec": {
            "machine_type": "n1-standard-4",
        },
        "container_spec": {"image_uri": "us-central1-docker.pkg.dev/{PROJECT_ID}/flower-
app/flower_image_distributed:multi_machine"}
    }
]
```

接著建立並執行 `CustomJob`，將 `staging_bucket` 中的 `{YOUR_BUCKET}` 替換為專案中的暫存值區。

```
my_custom_job = aiplatform.CustomJob(display_name='flowers-multi-worker',
                                     worker_pool_specs=worker_pool_specs,
                                     staging_bucket='gs://{YOUR_BUCKET}')

my_custom_job.run()
```

您可以在控制台中查看工作進度。

Vertex AI

- Dashboard
- Datasets
- Features
- Labelling tasks
- Workbench
- Pipelines
- Training**
- Experiments
- Models

Training [+ CREATE](#)

TRAINING PIPELINES **CUSTOM JOBS** HYPERPARAMETER TUNING JOBS

Custom jobs specify how Vertex AI runs your custom training code, including worker pools, machine types and settings related to your Python training application and custom container. Custom jobs are only used by custom-trained models and not AutoML models. [Learn more](#)

Region
us-central1 (Iowa) ▼ ⓘ

Filter Enter a property name

Name	ID	Status	Job type	Model
flowers-multi-worker	1521357130836213760	Pending	Custom job	—

🎉 恭喜！🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

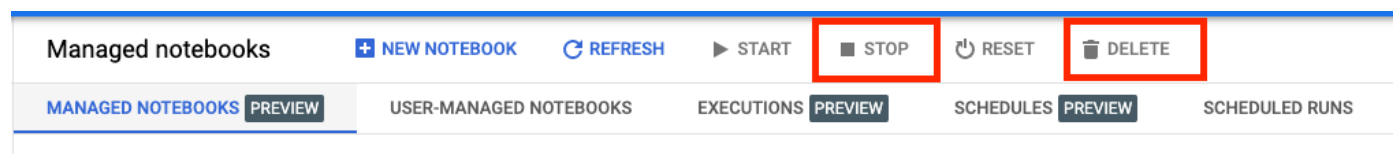
- 使用 TensorFlow 執行分散式訓練工作

如要進一步瞭解 Vertex 的其他部分，請參閱[說明文件](#)。

步驟 7 · 約 1 分鐘

7. 清除所用資源

由於我們將筆記本設定為在閒置 60 分鐘後逾時，因此不必擔心執行個體會關閉。如要手動關閉執行個體，請按一下控制台 Vertex AI Workbench 專區的「停止」按鈕。如要徹底刪除筆記本，請按一下「刪除」按鈕。



如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

